PEGASO
Università Telematica

TEMPLATE
PROJECT WORK

Corso di Studio	INFORMATICA PER LE AZIENDE DIGITALI (L-31)
Dimensione dell'elaborato	Minimo 6.000 – Massimo 10.000 parole *(pari a circa Minimo 12 – Massimo 20 pagine)*
Formato del file da caricare in piattaforma	PDF
Nome e Cognome	Gerardo Di Filippo
Numero di matricola	0312400533
Tema n. (Indicare il numero del tema scelto):	5
Titolo del tema (Indicare il titolo del tema scelto):	Machine Learning per Processi Aziendali
Traccia del PW n. (Indicare il numero della traccia scelta):	18
Titolo della traccia (Indicare il titolo della traccia scelta):	Triage automatico dei ticket con Machine Learning
Titolo dell'elaborato (Attribuire un titolo al proprio elaborato progettuale):	Smart Ticketing: classificazione e priorità automatica con Machine Learning

PARTE PRIMA – DESCRIZIONE DEL PROCESSO

Utilizzo delle conoscenze e abilità derivate dal percorso di studio
(Descrivere quali conoscenze e abilità apprese durante il percorso di studio sono state utilizzate per la redazione dell'elaborato, facendo eventualmente riferimento agli insegnamenti che hanno contribuito a maturarle):

1. Utilizzo delle conoscenze e abilità derivate dal percorso di studio

La realizzazione del progetto Smart Ticketing ha richiesto la mobilitazione integrata di competenze acquisite in più insegnamenti del percorso di studio in Informatica per le Aziende Digitali. Il lavoro ha costituito un banco di prova reale in cui concetti teorici apparentemente distanti hanno trovato applicazione concreta e sinergica, dimostrando quanto la formazione universitaria, anche quando si articola in moduli separati, prepari alla gestione di problemi complessi e multidimensionali.

1.1 Fondamenti di Programmazione e Ingegneria del Software

La struttura del progetto rispecchia i principi fondamentali dell'ingegneria del software appresi durante il percorso. I tre script Python che compongono il sistema — genera_dataset.py, pipeline_ml.py e genera_dashboard.py — sono stati progettati seguendo il principio della responsabilità singola (Single Responsibility Principle): ciascun modulo svolge una funzione precisa e ben delimitata, senza sovrapposizioni. Questa separazione facilita la manutenzione, il testing e la sostituzione di singoli componenti senza che le modifiche si propaghino a cascata sul resto del sistema.

L'utilizzo di funzioni helper modulari all'interno di ogni script (preprocess(), build_features(), build_pipeline(), evaluate(), top_words()) riflette il principio DRY (Don't Repeat Yourself),

anch'esso trattato nei corsi di programmazione. La pipeline di scikit-learn, che incapsula in un unico oggetto il trasformatore TF-IDF e il classificatore Naive Bayes, costituisce un esempio pratico del pattern Pipe-and-Filter, un pattern architetturale standard per sistemi di elaborazione dati.
La serializzazione dei modelli tramite il modulo pickle e la gestione dei file di configurazione in formato JSON mostrano dimestichezza con le pratiche di persistenza dei dati e di disaccoppiamento tra fasi di computazione, aspetti centrali nello sviluppo di applicazioni reali.

1.2 Basi di Dati e Gestione dei Dati Strutturati

Il corso di Basi di Dati ha fornito la concettualizzazione necessaria per progettare la struttura del dataset in modo coerente e normalizzato. Il file CSV tickets_sintetici.csv risponde a criteri di qualità del dato ben precisi: assenza di valori nulli, unicità dell'identificatore primario (id), dominio chiuso per i campi categorici (category e priority) e separazione chiara tra dati descrittivi (title, body) e metadati classificatori (category, priority).
La libreria pandas, utilizzata estensivamente nella pipeline ML per caricare, filtrare e manipolare il dataset, ha permesso di applicare le operazioni relazionali di base — selezione di colonne, filtraggio per condizione, join impliciti — in un contesto programmatico coerente con il modello mentale relazionale acquisito nei corsi di database. L'export finale in formato CSV del file predictions_batch.csv — contenente le predizioni affiancate ai dati originali — richiama il concetto di vista materializzata in ambito database, ovvero una rappresentazione derivata e precomputata dei dati sorgente.

1.3 Statistica e Probabilità Applicata

Il cuore teorico del progetto si basa sul Teorema di Bayes, approfondito nei corsi di Statistica e Probabilità. L'algoritmo Multinomial Naive Bayes stima, per ciascuna classe, la probabilità a posteriori di appartenenza data la sequenza di parole osservata nel testo. La comprensione della formula di Bayes, del concetto di likelihood e del ruolo dello smoothing di Laplace (parametro alpha) è stata indispensabile per configurare correttamente il modello e per interpretare i suoi output in modo critico, evitando di trattare le probabilità predette come valori assoluti.
Le metriche di valutazione adottate — Accuracy e F1-Score macro — sono radicate nella statistica inferenziale. In particolare, la scelta dell'F1-macro rispetto alla semplice accuracy riflette la consapevolezza del problema dello sbilanciamento delle classi: su un test set con distribuzione non uniforme delle priorità (13 ticket "alta", 32 "bassa", 15 "media"), la media ponderata avrebbe sovrastimato le performance reali del modello, mentre la media non pesata dell'F1 per classe fornisce una valutazione più equa e conservativa.
La matrice di confusione, strumento centrale nella valutazione dei classificatori, è stata prodotta e interpretata per entrambi i modelli. La lettura corretta della matrice — distinguendo falsi positivi, falsi negativi, precision e recall per singola classe — è un'abilità direttamente mutuata dai corsi di statistica e analisi dei dati.

1.4 Sistemi Informativi Aziendali

Il corso di Sistemi Informativi ha fornito il quadro concettuale per capire il contesto organizzativo in cui il sistema si inserisce. Il flusso di gestione dei ticket — ricezione, smistamento, presa in carico, risoluzione, chiusura — corrisponde a un processo aziendale strutturato (Business Process) che il sistema informatico è chiamato a supportare e parzialmente automatizzare. La distinzione tra i reparti Amministrazione, Tecnico e Commerciale come destinatari naturali dei flussi di comunicazione rispecchia la struttura funzionale tipica delle organizzazioni aziendali analizzata nei corsi di organizzazione d'impresa.
La progettazione della dashboard come interfaccia tra utente finale e logica computazionale sottostante segue il principio di astrazione degli strati informativi: l'operatore di help desk non deve conoscere i dettagli del modello ML per utilizzarlo efficacemente, esattamente come l'utente di un

ERP non gestisce direttamente le query SQL. Questa separazione tra livello di presentazione, livello applicativo e livello dati è un principio cardine dell'architettura dei sistemi informativi.

1.5 Tecnologie Web e Sviluppo Interfacce

La dashboard del progetto è stata realizzata in HTML5, CSS3 e JavaScript puro, senza dipendenze da framework o librerie esterne. Questa scelta, apparentemente minimalista, richiede in realtà una buona padronanza delle tecnologie web di base: gestione del DOM, event handling, manipolazione delle stringhe, rendering dinamico di tabelle e barre di progresso, esportazione di file lato client tramite Blob API.

La struttura a tab della dashboard, la palette cromatica coerente, la tipografia gerarchica e il sistema di badge colorati per le etichette di categoria e priorità riflettono i principi di User Interface Design e User Experience trattati nei corsi di sviluppo applicazioni. In particolare, la scelta del blu istituzionale come colore dominante e la gerarchia visiva che porta l'occhio dell'utente dalla categoria alla priorità e poi alle keyword seguono i principi Gestalt di prossimità e continuità, che orientano la percezione dell'informazione.

Fasi di lavoro e relativi tempi di implementazione per la predisposizione dell'elaborato
(Descrivere le attività svolte in corrispondenza di ciascuna fase di redazione dell'elaborato. Indicare il tempo dedicato alla realizzazione di ciascuna fase, le difficoltà incontrate e come sono state superate):

2. Fasi di lavoro e relativi tempi di implementazione

La predisposizione dell'elaborato ha seguito un processo iterativo suddiviso in sei fasi principali, ciascuna con obiettivi, attività e output ben definiti. I tempi indicati sono stime riferite al lavoro effettivo, esclusi i periodi di studio e approfondimento teorico.

Fase	Nome	Attività principali	Tempo stimato
1	Analisi e pianificazione	Studio del dominio, definizione requisiti, scelta tecnologica	4 ore
2	Progettazione dataset	Definizione template lessicali, regole keyword, schema CSV	5 ore
3	Implementazione generatore	Codifica genera_dataset.py, test di generazione, verifica distribuzione	3 ore
4	Pipeline ML e valutazione	Preprocessing, TF-IDF, Naive Bayes, metriche, grafici, serializzazione	6 ore
5	Dashboard e interfaccia	Progettazione UI, HTML/CSS/JS, integrazione grafici base64, export CSV	8 ore

6	Documentazione e report	Stesura relazione, README, revisione codice, commenti, testing finale	5 ore

2.1 Fase 1 – Analisi e pianificazione (4 ore)

La fase iniziale è stata dedicata alla comprensione del dominio applicativo e alla definizione puntuale dei requisiti funzionali e non funzionali. Sono stati analizzati i processi tipici di help desk aziendale per identificare le categorie di ticket più ricorrenti, le variabili linguistiche che le caratterizzano e i criteri operativi con cui gli operatori assegnano la priorità. Da questa analisi sono emersi i tre reparti destinatari (Amministrazione, Tecnico, Commerciale) e i tre livelli di priorità (alta, media, bassa), insieme ai segnali linguistici tipici di ciascun livello.

Parallelamente è avvenuta la valutazione tecnologica: il linguaggio Python è stato scelto per la sua ecosistema consolidato in ambito data science; scikit-learn per la disponibilità di TF-IDF e Naive Bayes in interfacce coerenti e ben documentate; la dashboard in HTML puro per evitare dipendenze da server come Streamlit o Gradio, rendendo il progetto eseguibile su qualsiasi macchina senza configurazioni aggiuntive.

2.2 Fase 2 – Progettazione del dataset (5 ore)

La progettazione del dataset sintetico è stata la fase concettualmente più impegnativa. Per ciascuna delle tre categorie sono stati scritti a mano 10 template di titolo e 10 template di descrizione, bilanciando realismo linguistico, varietà lessicale e separabilità tra classi. I template incorporano variabili dinamiche ({mese}, {num}, {importo}, {giorno}) che vengono riempite con valori casuali durante la generazione, producendo ticket sempre diversi a partire da un insieme ridotto di modelli.

Il sistema di regole per la priorità è stato calibrato empiricamente: le keyword di priorità alta corrispondono a situazioni di blocco totale o perdita di dati, le keyword di priorità media a disservizi parziali o ritardi, mentre l'assenza di segnali espliciti porta alla classificazione come bassa. La struttura gerarchica delle regole (alta prima di media) garantisce che i casi più gravi non vengano mai sottovalutati.

2.3 Fase 3 – Implementazione del generatore (3 ore)

La codifica del generatore ha trasformato i template progettati nella fase precedente in uno script Python eseguibile e riproducibile. L'uso del seed casuale fisso (random.seed(42)) garantisce la completa riproducibilità: eseguendo lo script sulle stesse condizioni si ottiene sempre lo stesso dataset. Il dataset viene mescolato dopo la generazione (shuffle) per evitare che i ticket siano raggruppati per categoria, simulando la distribuzione casuale tipica di un sistema di ticketing reale. Infine, l'identificatore id viene riassegnato progressivamente dopo lo shuffle per mantenere la coerenza degli indici.

2.4 Fase 4 – Pipeline ML e valutazione (6 ore)

Questa fase ha richiesto il maggior impegno tecnico. Sono stati implementati in sequenza il preprocessing testuale, la costruzione della pipeline TF-IDF + Naive Bayes, il training su set di addestramento, la predizione sul test set e il calcolo delle metriche. Particolare attenzione è stata dedicata alla scelta dei parametri del TF-IDF (ngram_range, max_features, sublinear_tf) e allo smoothing del Naive Bayes (alpha), verificando l'impatto di ciascun parametro sulle metriche di test attraverso esperimenti manuali.

La funzione top_words() è stata sviluppata per estrarre le parole più discriminanti per categoria, sfruttando i log-probabilità condizionali appresi dal classificatore. I grafici di confusion matrix e F1-score sono stati prodotti con matplotlib e seaborn, salvati come PNG ad alta risoluzione e successivamente incorporati sia nel report Word sia nella dashboard HTML.

2.5 Fase 5 – Dashboard e interfaccia (8 ore)

La fase più lunga del progetto ha riguardato la progettazione e lo sviluppo della dashboard interattiva. La scelta di realizzarla come file HTML standalone, con tutti gli asset (grafici, dati) incorporati in base64 o JSON inline, ha richiesto una progettazione attenta per evitare dipendenze esterne mantenendo comunque una buona esperienza utente. Il design a quattro tab (Classifica Ticket, Batch & Risultati, Metriche, Dataset) è stato scelto per separare le funzionalità senza sovraccaricare l'interfaccia.

La logica di classificazione lato browser è stata implementata in JavaScript puro come simulazione semplificata del modello Python, basata sullo stesso sistema di keyword rules. Questa duplicazione è stata necessaria perché un file HTML statico non può eseguire codice Python; la soluzione adottata rappresenta un compromesso pragmatico che mantiene il prototipo completamente autonomo.

2.6 Fase 6 – Documentazione e report (5 ore)

L'ultima fase è stata dedicata alla redazione della documentazione tecnica: il file README.md con istruzioni di installazione e riproduzione, i commenti inline nel codice e la presente relazione. La stesura del report ha richiesto la rielaborazione critica di tutte le scelte progettuali, trasformando decisioni tecniche in argomentazioni motivate accessibili anche a lettori non specialisti.

Risorse e strumenti impiegati

(Descrivere quali risorse - bibliografia, banche dati, ecc. - e strumenti - software, modelli teorici, ecc. - sono stati individuati ed utilizzati per la redazione dell'elaborato. Descrivere, inoltre, i motivi che hanno orientato la scelta delle risorse e degli strumenti, la modalità di individuazione e reperimento delle risorse e degli strumenti, le eventuali difficoltà affrontate nell'individuazione e nell'utilizzo di risorse e strumenti ed il modo in cui sono state superate):

3. Risorse e strumenti impiegati

3.1 Linguaggi e ambienti di sviluppo

Il progetto è stato sviluppato interamente in Python 3.10+, scelto per la sua maturità nell'ecosistema data science e per la semplicità sintattica che ne facilita la lettura e la manutenzione. La compatibilità con Python 3.10 è garantita dall'utilizzo di type hints moderni (list[dict], list[str]) e dell'operatore walrus, introdotti rispettivamente nelle versioni 3.9 e 3.8. L'ambiente di esecuzione può essere un qualsiasi interprete Python installato localmente, senza necessità di container Docker o ambienti cloud.
La dashboard è stata realizzata con le tecnologie web standard: HTML5 per la struttura, CSS3 per la presentazione e JavaScript ES6+ per l'interattività. La scelta di non utilizzare framework come React, Vue o Angular è deliberata: riduce la complessità di setup e rende il file comprensibile a chiunque abbia competenze web di base.

3.2 Librerie Python

Libreria	Versione	Utilizzo nel progetto
scikit-learn	1.3+	TfidfVectorizer, MultinomialNB, Pipeline, train_test_split, metriche
pandas	2.0+	Caricamento CSV, manipolazione DataFrame, export predizioni
numpy	1.24+	Ordinamento argomenti per top_words(), operazioni vettoriali
matplotlib	3.7+	Generazione grafici PNG (confusion matrix, bar chart F1)
seaborn	0.12+	Heatmap per la confusion matrix con annotazioni numeriche
pickle (stdlib)	built-in	Serializzazione e deserializzazione dei modelli ML
json (stdlib)	built-in	Persistenza di metriche e top_words in formato leggibile
re (stdlib)	built-in	Espressioni regolari per il preprocessing testuale
base64 (stdlib)	built-in	Encoding delle immagini PNG per incorporarle nella dashboard HTML

3.3 Strumenti di sviluppo e versionamento

Il codice è stato sviluppato con un editor di testo standard e organizzato in un repository con struttura di directory convenzionale. Il versionamento tramite Git

garantisce la tracciabilità delle modifiche e la possibilità di ritornare a versioni precedenti in caso di errori. Il file README.md segue le convenzioni Markdown standard di GitHub/GitLab, con badge di stato, struttura ad albero del repository e istruzioni di installazione graduate.

Per la generazione del report Word è stato utilizzato il pacchetto Node.js docx (versione 9.5+), che permette la produzione programmatica di documenti .docx rispettando lo standard OOXML. Questa scelta consente di generare la relazione con dati aggiornati automaticamente (metriche, grafici) semplicemente rieseguendo lo script, senza dover aggiornare manualmente un documento Word.

3.4 Risorse di conoscenza

Le risorse teoriche consultate durante lo sviluppo includono la documentazione ufficiale di scikit-learn, in particolare le sezioni dedicate a TfidfVectorizer e MultinomialNB, la documentazione di pandas per le operazioni di manipolazione dati e la letteratura accademica di riferimento sul Natural Language Processing per la classificazione di testi brevi. Per la progettazione dell'interfaccia si è fatto riferimento alle linee guida di accessibilità WCAG 2.1 e ai principi di design system consolidati nella pratica professionale.

PARTE SECONDA – PREDISPOSIZIONE DELL'ELABORATO

Obiettivi del progetto
(Descrivere gli obiettivi raggiunti dall'elaborato, indicando in che modo esso risponde a quanto richiesto dalla traccia):

4. Obiettivi del progetto

4.1 Obiettivo generale

L'obiettivo generale del progetto è dimostrare la fattibilità di un sistema automatico di classificazione e prioritizzazione dei ticket aziendali basato su tecniche di Machine Learning, realizzato con strumenti open source standard e concepito per essere completamente riproducibile da chiunque disponga di Python e delle librerie indicate. Il progetto non ambisce a costruire un sistema pronto per la produzione, ma a costituire un prototipo funzionante e documentato che possa essere validato, esteso e adattato a contesti reali.

4.2 Obiettivi specifici

- Costruire un dataset sintetico di almeno 200 ticket testuali etichettati per categoria (Amministrazione, Tecnico, Commerciale) e priorità (alta, media, bassa), senza ricorrere a dati personali reali.
- Addestrare almeno un modello di classificazione testuale in grado di assegnare automaticamente la categoria corretta a nuovi ticket non visti durante il training.
- Addestrare un secondo modello per la stima della priorità operativa, utilizzando le stesse feature testuali del modello di categoria.

- Valutare entrambi i modelli con metriche rigorose (accuracy e F1-macro) su un test set separato (split 80/20), producendo i relativi grafici di confusion matrix.
- Realizzare una dashboard interattiva che permetta all'operatore di inserire un ticket e ricevere immediatamente categoria predetta, priorità e le cinque parole più influenti nella decisione.
- Produrre un file CSV con le predizioni batch sull'intero dataset, simulando il flusso di smistamento automatico di un help desk.
- Documentare il progetto con un README chiaro, commenti nel codice e una relazione tecnica completa, rendendo l'elaborato riproducibile in pochi passaggi.

4.3 Obiettivi di qualità del codice

Accanto agli obiettivi funzionali, il progetto si è dato obiettivi espliciti di qualità del codice: leggibilità (funzioni brevi, nomi esplicativi, commenti pertinenti), modularità (ogni script ha una responsabilità unica), riproducibilità (seed fisso, nessuna dipendenza da servizi esterni), portabilità (nessun percorso assoluto, struttura di directory relativa) e manutenibilità (parametri configurabili in cima agli script, nessun valore magico inline).

Contestualizzazione
(Descrivere il contesto teorico e quello applicativo dell'elaborato realizzato):

5. Contestualizzazione

5.1 Il problema dello smistamento manuale dei ticket

In qualsiasi organizzazione di medie o grandi dimensioni, la gestione delle richieste di assistenza interna (help desk) rappresenta un processo ad alto volume e frequentemente un collo di bottiglia operativo. Le richieste arrivano da dipendenti, clienti o partner attraverso canali eterogenei — email, moduli web, chat aziendali, telefonate trascritte — e devono essere lette, classificate per reparto di competenza e ordinate per urgenza prima di poter essere prese in carico.

Quando questo processo viene svolto manualmente, emergono tre criticità sistematiche. La prima è la latenza: ogni ticket deve aspettare che un operatore umano lo legga e lo smisti, il che introduce un ritardo che può variare da pochi minuti a diverse ore nelle fasi di picco. La seconda è l'inconsistenza: operatori diversi possono classificare lo stesso tipo di ticket in modo diverso, rendendo i dati storici poco affidabili per analisi successive. La terza è il rischio di sottostima: un ticket critico che arriva durante un momento di sovraccarico può essere trattato come ordinario, con potenziali conseguenze operative significative.

5.2 Il Natural Language Processing come soluzione

Il Natural Language Processing (NLP) offre strumenti consolidati per automatizzare la lettura e la classificazione del testo non strutturato. La Text Classification — il task di assegnare un testo a una o più categorie predefinite — è uno dei problemi più

studiati in NLP, con decenni di letteratura e benchmark disponibili. Le tecniche che vanno dai semplici modelli bag-of-words (TF-IDF + Naive Bayes o SVM) fino ai Transformer moderni (BERT, RoBERTa) offrono un ventaglio di soluzioni scalabili in base alle risorse disponibili e alla complessità richiesta.
Per un prototipo universitario orientato alla comprensibilità e alla riproducibilità, l'approccio TF-IDF + Naive Bayes rappresenta il punto di partenza ideale: è matematicamente trasparente, computazionalmente economico, interpretabile (le feature sono parole leggibili dall'uomo) e sorprendentemente efficace su testi brevi con lessico sufficientemente specifico per dominio.

5.3 Posizionamento rispetto alla letteratura e alle soluzioni commerciali

Il problema affrontato da Smart Ticketing è ben noto nel settore IT Service Management (ITSM). Soluzioni commerciali come ServiceNow, Zendesk e Jira Service Management offrono funzionalità di smistamento automatico basate su ML come feature premium. Tuttavia, queste soluzioni sono chiuse, costose e non trasparenti nei loro meccanismi interni. Il valore del presente prototipo sta precisamente nell'opposto: tutto il codice è aperto, ogni decisione è documentata e giustificata, e il sistema può essere ispezionato, modificato e ri-addestrato da chiunque abbia competenze di base in Python e scikit-learn.
Questo posizionamento è coerente con la filosofia della formazione universitaria in informatica applicata: non si tratta di replicare soluzioni commerciali, ma di capirne i meccanismi fondamentali costruendole da zero, in modo da poter un giorno valutare criticamente le soluzioni esistenti e proporre miglioramenti consapevoli.

5.4 Contesto normativo e privacy

Un aspetto rilevante nella contestualizzazione del progetto è la dimensione normativa. I sistemi di ticketing reali trattano dati personali (nome del richiedente, reparto, contenuto della comunicazione) che rientrano nel perimetro del GDPR. La scelta di utilizzare esclusivamente dati sintetici, privi di qualsiasi informazione personale identificabile, elimina questa problematica per il prototipo e costituisce al contempo un modello di approccio privacy-by-design: il sistema funziona senza aver mai bisogno di accedere a dati sensibili, limitandosi al testo del ticket depurato di qualsiasi riferimento personale.

Descrizione dei principali aspetti progettuali
(Sviluppare l'elaborato richiesto dalla traccia prescelta):

6. Descrizione dei principali aspetti progettuali

6.1 Architettura del sistema

Il sistema è composto da tre strati funzionali distinti, comunicanti attraverso file intermediari in formato standard (CSV, JSON, PNG, PKL):

- Strato dati: il generatore sintetico produce il dataset grezzo in formato CSV. Questo file costituisce l'unico input della pipeline ML e può essere sostituito in qualsiasi momento con dati reali senza modificare il resto del sistema.

- Strato elaborazione: la pipeline ML legge il dataset, esegue il preprocessing e il training, produce i modelli serializzati, le metriche in JSON, i grafici in PNG e il CSV di predizioni batch.
- Strato presentazione: il generatore della dashboard legge i modelli, i JSON e le immagini PNG per produrre un file HTML autocontenuto che incorpora tutto il necessario per l'interazione utente.

Questa architettura a strati disaccoppiati garantisce che ciascun componente possa essere sviluppato, testato e sostituito indipendentemente dagli altri. In un'evoluzione futura, lo strato dati potrebbe essere collegato a un database aziendale reale, lo strato elaborazione potrebbe essere sostituito da un modello più sofisticato e lo strato presentazione potrebbe diventare un'applicazione web con backend Flask o FastAPI, senza che nessuna di queste modifiche richieda di toccare gli altri strati.

6.2 Generazione del dataset sintetico

Il generatore implementa un sistema di template parametrizzati con una logica di riempimento casuale. Ogni template contiene segnaposto ({mese}, {num}, {importo}, {giorno}) che vengono sostituiti con valori estratti casualmente da liste predefinite durante la generazione. Questo meccanismo produce ticket sempre diversi a partire da un insieme compatto di modelli, garantendo varietà senza richiedere la redazione manuale di centinaia di testi.

La distribuzione delle priorità nel dataset generato (bassa: 133, media: 98, alta: 69 su 300 ticket totali) rispecchia una distribuzione realistica: nella maggior parte degli ambienti aziendali, i ticket urgenti rappresentano la minoranza, mentre la maggioranza riguarda richieste ordinarie. Questa distribuzione sbilanciata rende il task di predizione della priorità più realistico e più difficile rispetto a una distribuzione uniforme, ed è il motivo principale per cui il modello di priorità raggiunge prestazioni inferiori rispetto a quello di categoria.

6.3 Preprocessing testuale

La funzione preprocess() implementa una pipeline di normalizzazione in tre passi: conversione in minuscolo, rimozione di tutti i caratteri non appartenenti all'alfabeto italiano (incluse le lettere accentate, gestite dalla classe di caratteri estesa nell'espressione regolare), e compressione degli spazi multipli. Il testo normalizzato viene poi concatenato con il titolo del ticket prima della vettorizzazione.

La scelta di non applicare la rimozione delle stopword è stata deliberata: in italiano, articoli e preposizioni contribuiscono raramente alla classificazione categorica, ma la loro rimozione richiede una lista curata per il dominio specifico. In assenza di tale lista, la rimozione rischia di introdurre più rumore che beneficio. Questo è uno dei limiti esplicitamente riconosciuti del prototipo, che in una versione avanzata verrebbe risolto con la libreria NLTK o spaCy.

6.4 Vettorizzazione TF-IDF

La trasformazione del testo in vettori numerici avviene tramite TF-IDF (Term Frequency–Inverse Document Frequency), una rappresentazione che assegna a ogni parola un peso proporzionale alla sua frequenza nel documento e inversamente proporzionale alla sua frequenza nel corpus. Le parole molto comuni in tutti i documenti (come articoli o congiunzioni) ricevono un peso basso; le parole specifiche di pochi documenti ricevono un peso alto.

Il parametro ngram_range=(1,2) estende il vocabolario ai bigrammi: coppie di parole consecutive come "schermata blu", "nota credito" o "errore critico" vengono trattate come feature unitarie, catturando il significato composito che i singoli unigrammi non possono esprimere. Questo è particolarmente importante per la classificazione della priorità, dove espressioni come "errore critico" sono molto più informative della singola parola "errore". Il parametro sublinear_tf=True applica la trasformazione log(1+tf) invece della semplice frequenza, smorzando l'influenza di termini molto ripetuti e migliorando la robustezza su testi di lunghezza variabile.

6.5 Modello di classificazione: Multinomial Naive Bayes

L'algoritmo Multinomial Naive Bayes (MNB) è particolarmente adatto alla classificazione di testi rappresentati come bag-of-words o TF-IDF. Il modello stima la probabilità a posteriori di ciascuna classe dato il vettore di feature, applicando il Teorema di Bayes con l'assunzione di indipendenza condizionale tra le feature (l'assunzione "naive"). Nonostante questa assunzione sia raramente soddisfatta nel testo naturale, l'MNB produce risultati competitivi su molti task di text classification, specialmente su corpus di dimensioni ridotte.

Il parametro alpha=0.5 controlla lo smoothing di Laplace: invece di assegnare probabilità zero alle parole non viste durante il training (il che renderebbe impossibile qualsiasi predizione per testi contenenti parole nuove), si aggiunge un piccolo valore pseudocount. Il valore 0.5 è stato scelto come compromesso tra lo smoothing completo (alpha=1, smoothing di Laplace classico) e l'assenza di smoothing (alpha=0): sperimentando valori diversi, alpha=0.5 ha mostrato F1-score leggermente superiori sul test set di priorità.

6.6 Strategia di valutazione

La valutazione dei modelli segue il protocollo standard per la classificazione supervisionata: uno split train/test in proporzione 80/20 (240 campioni di training, 60 di test) con stratificazione per categoria, che garantisce che la proporzione tra le classi sia preservata in entrambi i set. Il parametro random_state=42 garantisce la riproducibilità dello split.

Le metriche adottate sono l'Accuracy (percentuale di predizioni corrette sull'intero test set) e l'F1-Score macro (media non pesata dell'F1 per classe), che penalizza equamente gli errori su ogni classe indipendentemente dalla sua dimensione. L'F1 è la media armonica di precision e recall: precision misura quante delle predizioni positive sono effettivamente corrette; recall misura quante delle istanze positive reali

sono state individuate. La media armonica penalizza gli squilibri tra le due misure, rendendo l'F1 più informativo della semplice media aritmetica.

6.7 Interpretabilità: le top-5 parole per categoria

Un aspetto progettuale spesso trascurato nei sistemi di ML è l'interpretabilità: la capacità di spiegare perché il modello ha preso una determinata decisione. In Smart Ticketing, l'interpretabilità è implementata tramite la funzione top_words(), che estrae le cinque feature con il log-probabilità condizionale più alto per ciascuna categoria appresa dal classificatore Naive Bayes. Queste parole rappresentano i segnali linguistici che il modello considera più discriminanti per quella categoria.

La visualizzazione delle top-5 parole nella dashboard permette all'operatore di help desk di verificare rapidamente se la predizione è sensata o se il modello si è basato su segnali anomali. Questa trasparenza riduce la resistenza all'adozione del sistema: l'utente non deve fidarsi ciecamente di un output opaco, ma può vedere la "motivazione" della predizione e decidere se accettarla o sovrascriverla.

6.8 Dashboard HTML standalone

La scelta di realizzare la dashboard come file HTML standalone — senza server, senza framework, senza dipendenze da installare — è una delle decisioni progettuali più significative del progetto. Questa scelta risolve il problema della portabilità: il file dashboard.html può essere inviato via email, condiviso su Google Drive o aperto da una chiavetta USB, funzionando su qualsiasi computer con un browser moderno senza alcuna configurazione.

I grafici di confusione e il bar chart dell'F1-score vengono incorporati nel file HTML come stringhe base64, eliminando le dipendenze da file esterni. I dati del dataset e le predizioni batch vengono inclusi come oggetti JavaScript inline. La classificazione lato browser è implementata come replica semplificata della logica Python, basata sullo stesso sistema di keyword rules: questa duplicazione è il prezzo da pagare per la completa autonomia del file HTML.

La struttura a quattro tab risponde a diverse esigenze degli utenti: la tab "Classifica Ticket" per l'uso quotidiano dell'operatore, la tab "Batch & Risultati" per l'analisi di volumi di ticket, la tab "Metriche del Modello" per la valutazione tecnica del sistema e la tab "Dataset Sintetico" per la comprensione dei dati di addestramento.

Campi di applicazione
(Descrivere gli ambiti di applicazione dell'elaborato progettuale e i vantaggi derivanti della sua applicazione):

7. Campi di applicazione

7.1 Help desk aziendale interno

Il campo di applicazione primario e naturale del sistema è l'help desk aziendale interno: quella struttura che gestisce le richieste di supporto provenienti dai dipendenti dell'organizzazione verso i reparti di servizio (IT, Amministrazione, HR,

Facilities). In questo contesto, il sistema automatizza lo smistamento dei ticket verso il reparto competente e segnala quelli prioritari per un'attenzione immediata, liberando gli operatori di front-line dal lavoro ripetitivo di lettura e categorizzazione per concentrarsi sulla risoluzione effettiva dei problemi.

Un'azienda con 200 dipendenti riceve mediamente tra i 15 e i 40 ticket al giorno. Con tempi di smistamento manuale di 2-5 minuti per ticket, il processo assorbe tra 30 minuti e 3 ore di lavoro quotidiano. Un sistema automatico che gestisce il 90% dei ticket ordinari con precisione sufficiente può ridurre questo tempo a pochi minuti di supervisione, con ricadute significative sulla produttività operativa.

7.2 Customer support e assistenza clienti

Un secondo campo di applicazione immediato è il customer support: il sistema che gestisce le richieste di assistenza provenienti dai clienti esterni verso l'azienda. In questo contesto, le categorie potrebbero essere ridefinite per rispecchiare la struttura del servizio di supporto (pre-vendita, post-vendita, fatturazione, assistenza tecnica) e le priorità potrebbero incorporare variabili aggiuntive come il livello contrattuale del cliente (SLA) o il valore dell'account.

Molte piattaforme di e-commerce e software as a service ricevono centinaia o migliaia di richieste giornaliere. In questi ambienti, anche un sistema di classificazione con accuracy dell'80% rappresenta un vantaggio operativo significativo rispetto allo smistamento manuale, purché sia integrato con un meccanismo di revisione umana per i casi incerti.

7.3 Pubblica amministrazione

Le pubbliche amministrazioni gestiscono volumi crescenti di richieste dei cittadini attraverso sportelli digitali, PEC e moduli online. Un sistema di classificazione automatica potrebbe smistare le pratiche verso gli uffici competenti (tributi, anagrafe, edilizia, servizi sociali) e segnalare le urgenze (ricorsi in scadenza, richieste con termini perentori) per un trattamento prioritario. In questo contesto, la trasparenza del modello — la possibilità di spiegare perché una richiesta è stata classificata in un certo modo — è particolarmente importante per ragioni di accountability pubblica.

7.4 Gestione delle segnalazioni di conformità

In ambito di compliance aziendale (GDPR, sicurezza sul lavoro, antiriciclaggio), le organizzazioni ricevono segnalazioni e rapporti che devono essere classificati per tipo di violazione o rischio e prioritizzati per gravità. Un sistema di classificazione testuale potrebbe automatizzare il primo screening di queste segnalazioni, indirizzandole verso il responsabile della conformità appropriato e segnalando quelle che richiedono escalation immediata.

7.5 Ricerca e sviluppo accademico

Dal punto di vista accademico, il progetto costituisce un caso di studio completo e riproducibile per l'insegnamento di tecniche NLP di base in corsi universitari. La sua struttura modulare, la documentazione estesa e la dashboard interattiva lo rendono

un materiale didattico di valore per illustrare il flusso completo di un progetto di text classification: dalla generazione dei dati al deployment di un'interfaccia utente, passando per training, valutazione e interpretazione dei risultati.

7.6 Monitoraggio dei social media e delle recensioni

Con adattamenti al dataset di addestramento, lo stesso framework può essere applicato alla classificazione di commenti sui social media, recensioni di prodotti o feedback di soddisfazione. In questi contesti, le categorie potrebbero essere sentimenti (positivo, negativo, neutro) o temi (qualità del prodotto, spedizione, assistenza), e le priorità potrebbero indicare la necessità di una risposta urgente (recensioni molto negative su piattaforme pubbliche, commenti con alto engagement).

Valutazione dei risultati
(Descrivere le potenzialità e i limiti ai quali i risultati dell'elaborato sono potenzialmente esposti)

8. Valutazione dei risultati

8.1 Metriche ottenute e loro interpretazione

I due modelli prodotti dal sistema mostrano performance significativamente diverse, e questa differenza è essa stessa un risultato informativo importante del progetto.

Modello	Accuracy	F1-macro	Interpretazione
Categoria	**100.0%**	**1.000**	**Separabilità lessicale completa**
Priorità	**81.7%**	**0.814**	**Confusione tra alta e media**

Il modello di categoria raggiunge un'accuracy del 100% e un F1-macro di 1.000. Questo risultato, apparentemente straordinario, è correttamente interpretabile come una conferma della buona progettazione del dataset sintetico: i template lessicali per le tre categorie sono stati costruiti con vocabolari sufficientemente separati da rendere il problema di classificazione trivialmente risolvibile per TF-IDF + Naive Bayes. Questa performance non è generalizzabile a dati reali, dove la distinzione tra categorie è molto più sfumata, ma valida il corretto funzionamento della pipeline tecnica.

Il modello di priorità raggiunge un'accuracy dell'81.7% con F1-macro di 0.814. Questo risultato è più realistico e informativo: le keyword di priorità sono trasversali alle categorie ("bloccato" può comparire in ticket tecnici, amministrativi o commerciali) e la distribuzione sbilanciata delle classi (69 alta, 98 media, 133

bassa) rende il task genuinamente più difficile. L'errore principale si concentra sulla classe "alta", che mostra recall più basso: alcuni ticket urgenti vengono classificati come "media". In un contesto reale, questo tipo di errore è il più costoso, ed è per questo che la sezione sui limiti del prototipo raccomanda l'implementazione di una soglia di confidenza che permetta di ridurre il numero di falsi negativi sulla classe alta, anche a costo di aumentare i falsi positivi.

8.2 Analisi per classe: F1, Precision e Recall

L'analisi per classe del modello di priorità rivela un pattern significativo:

- Classe "alta" (13 campioni nel test set): precision 0.67, recall 0.92, F1 0.77. Il recall alto indica che la maggior parte dei ticket urgenti viene correttamente identificata; la precision bassa indica che alcuni ticket non urgenti vengono erroneamente classificati come alta. Questo è il tipo di errore meno pericoloso in un contesto operativo.
- Classe "bassa" (32 campioni nel test set): precision 0.92, recall 0.75, F1 0.83. La precision alta indica pochi falsi allarmi; il recall moderato indica che alcuni ticket a bassa priorità vengono classificati come media o alta.
- Classe "media" (15 campioni nel test set): precision 0.81, recall 0.87, F1 0.84. Buone prestazioni su entrambe le dimensioni, con leggera tendenza a confondere la media con la bassa.

Il pattern complessivo mostra che il modello tende a essere "prudente": in caso di incertezza, propende verso priorità più alta piuttosto che più bassa. Questo comportamento è operativamente desiderabile in un help desk: meglio trattare con urgenza un ticket che non la richiedeva che ignorare un problema critico.

8.3 Valutazione della dashboard

La dashboard supera gli obiettivi funzionali stabiliti: permette di classificare un ticket in tempo reale, mostra le cinque parole più influenti, visualizza le barre di confidenza per categoria e priorità, consente di caricare esempi rapidi predefiniti, mostra le predizioni su un batch di 20 ticket con indicatori di correttezza e permette di esportare i risultati in CSV. L'interfaccia è responsive, accessibile su browser desktop e mobile, e caricabile senza connessione internet.

Il limite principale della dashboard è che la logica di classificazione lato browser è una replica semplificata del modello Python, non il modello esatto. Le predizioni JavaScript possono differire marginalmente da quelle Python in casi limite, dove la singola keyword non è sufficiente a determinare univocamente categoria e priorità. In una versione produzione, questo problema verrebbe risolto espondo il modello Python tramite un'API REST (Flask o FastAPI), con la dashboard che effettua chiamate HTTP invece di classificare localmente.

8.4 Valutazione dell'architettura complessiva

L'architettura a tre script indipendenti si è dimostrata efficace durante lo sviluppo: la possibilità di rieseguire singoli step senza invalidare gli altri ha accelerato

significativamente il processo di debugging e ottimizzazione. La comunicazione tramite file standard (CSV, JSON, PNG, PKL) ha permesso di verificare facilmente l'output di ogni stadio senza dover strumentare il codice con log aggiuntivi.
Il tempo totale di esecuzione dell'intera pipeline (genera_dataset.py + pipeline_ml.py + genera_dashboard.py) è inferiore a 10 secondi su qualsiasi laptop moderno, rendendo il sistema completamente praticamente istantaneo da un punto di vista dell'utilizzo.

8.5 Considerazioni critiche e onestà intellettuale

Una valutazione completa dei risultati richiede onestà intellettuale riguardo ai limiti strutturali del prototipo. L'accuracy del 100% sul modello di categoria non è un traguardo tecnologico: è una conseguenza diretta della progettazione sintetica del dataset. Se il modello venisse applicato a ticket reali, la performance crollerebbe a valori molto più bassi, probabilmente tra il 60% e l'80% a seconda della qualità e della varietà del testo reale.
Questo non significa che il progetto abbia fallito il suo obiettivo: il prototipo dimostra correttamente il flusso tecnico completo, dall'ingestion dei dati al deployment dell'interfaccia. La precisione delle metriche su dati sintetici è un artefatto controllato e dichiarato, non un tentativo di sopravvalutare le capacità reali del sistema. In questo senso, il progetto raggiunge pienamente l'obiettivo dichiarato di essere un prototipo essenziale e riproducibile, non un sistema pronto per la produzione.
Il valore aggiunto più significativo del progetto non risiede nelle metriche numeriche, ma nella completezza e nella coerenza del flusso implementato: un dataset generato con criteri espliciti e documentati, una pipeline di training configurabile e riproducibile, una valutazione rigorosa con metriche standard e una interfaccia utente accessibile e funzionante. Questi elementi costituiscono una base solida su cui innestare i miglioramenti identificati nella sezione sui limiti del sistema, avvicinandolo progressivamente a un'applicazione reale.

8.6 Potenziale di sviluppo

I risultati ottenuti suggeriscono tre direttrici prioritarie per sviluppi futuri. La prima è la raccolta di dati reali: anche un dataset di 500-1000 ticket anonimi e storici aumenterebbe drammaticamente la generalizzabilità del modello, riducendo l'artefatto della separabilità lessicale artificiale. La seconda è l'integrazione di tecniche NLP più avanzate: la lemmatizzazione con spaCy e la rimozione delle stopword italiane migliorerebbero il preprocessing senza aumentare significativamente la complessità del codice. La terza è l'introduzione di soglie di confidenza: prevedere una classe "incerto" quando la probabilità massima scende sotto una soglia configurabile (es. 60%) ridurrebbe gli errori ad alto impatto, indirizzando verso la revisione umana i casi ambigui invece di classificarli forzosamente.

A lungo termine, la sostituzione di Naive Bayes con un modello Transformer pre-addestrato sull'italiano (come BERT-italiano o il più recente Camoscio) potrebbe portare il sistema a performance paragonabili a quelle delle soluzioni commerciali, mantenendo comunque la trasparenza e la controllabilità che caratterizzano questo progetto.

Note conclusive

Il progetto Smart Ticketing ha raggiunto tutti gli obiettivi prefissati, producendo un sistema funzionante, documentato e riproducibile per la classificazione automatica dei ticket aziendali. Il percorso di sviluppo ha richiesto la mobilitazione integrata di competenze acquisite in più insegnamenti del corso di studi — dalla programmazione alla statistica, dai sistemi informativi alle tecnologie web — dimostrando come la formazione universitaria, quando applicata a un problema concreto, generi soluzioni tecnicamente solide e contestualmente rilevanti.

Il progetto ha anche dimostrato l'importanza di mantenere un equilibrio tra ambizione tecnica e realismo: scegliere gli strumenti giusti per il contesto giusto, riconoscere apertamente i limiti del proprio lavoro e tracciare chiaramente il percorso verso versioni più avanzate sono competenze professionali fondamentali quanto la padronanza degli algoritmi. In questo senso, Smart Ticketing è non solo un prototipo funzionante, ma anche un esercizio di ingegneria del software responsabile e consapevole.